CRBench: A Method-Agnostic, Resource-Aware Evaluation Framework for Long-Context Large Language Models*

Anonymous authors
Paper under double-blind review

Abstract

As Large Language Models (LLMs) expand context windows beyond 32K, 128K, and 1M tokens, the Key-Value (KV) cache becomes a primary operational bottleneck during autoregressive inference. While diverse compression paradigms have emerged—including quantization, token eviction, pooling, low-rank projection, and factorized representations—current evaluation practices often separate task accuracy from physical resource usage and report results across heterogeneous tasks, models, and accounting conventions.

We introduce CRBench, a method-agnostic, resource-aware evaluation framework for rigorously benchmarking long-context LLMs and context-compression techniques. CRBench formulates evaluation around an atomic query-level primitive that pairs every compressed execution with an uncompressed dense baseline on the identical prompt. The framework decouples evaluation into two complementary tracks: Part 1 measures algorithmic representation utility ($\mathcal{S}_{\text{res}}$) by combining absolute task quality with effective memory savings, where effective bits-per-element (b_{eff}) accounts for representation metadata and storage overheads; Part 2 measures hardware system serving utility ($\mathcal{S}_{\text{sys}}$) using physical time-to-first-token (TTFT), decode throughput, peak VRAM, and a baseline-relative hardware efficiency multiplier.

We evaluate seven context-compression methods alongside an uncompressed dense reference across three foundation models (Gemma 4 E2B, Gemma 4 E4B, and Qwen2.5 7B), spanning 3,150 queries across five long-context tasks from 2K to 128K tokens on NVIDIA RTX hardware. Empirical results show that: (1) KIVI-style INT4 quantization achieves the highest cross-model system score (mean $\mathcal{S}_{\text{sys}} = 70.3$) with near-baseline hardware efficiency ($\phi \geq 0.96\times$); (2) SnapKV provides strong algorithmic utility ($\mathcal{S}_{\text{res}} = 60.8$ on Gemma 4 E4B) but incurs substantial prefill overhead under un-fused attention pooling (TTFT = 83.4 s at 32K); (3) DKV-Mid operates at 5.00–5.84 effective bits-per-element across the three models, with 44.7–55.3% mean task accuracy; and (4) all evaluated methods, including dense FP16, perform near zero on multi-step variable tracking, suggesting that this task is substantially limited by the underlying base models rather than KV compression. CRBench is released with an extensible adapter API and reproducible evaluation pipeline.

*Large language models were used as assistive writing tools for stylistic editing and drafting under the authors' direct supervision; the authors take full responsibility for all content, experimental methodology, and scientific claims.

1 Introduction

Modern Large Language Models (LLMs) have achieved remarkable breakthroughs across multi-document synthesis, complex multi-hop question answering, repository-scale code generation, and autonomous agent workflows (Brown et al., 2020; Achiam et al., 2023; Touvron et al., 2023; Bai et al., 2023a). These capabilities rely on long-context processing, expanding context horizons from 2,048 tokens to 32,768, 131,072, and beyond (Reid et al., 2024; DeepSeek-AI, 2024).

However, contextual expansion is fundamentally bounded by the memory footprint of the Key-Value (KV) cache in standard Transformer self-attention architectures (Vaswani et al., 2017). Because standard attention caches intermediate key and value tensor representations for every historical token across all layers and attention heads, KV cache memory scales linearly with sequence length L and batch size B:

$$M_{\text{KV}} = 2 \times N_{\text{layers}} \times N_{\text{kv_heads}} \times d_{\text{head}} \times L \times B \times \text{sizeof(dtype)} \quad \text{[bytes]} \tag{1}$$

For a representative 8-billion parameter model (e.g., Llama-3.1-8B with 32 layers, 8 Key-Value heads under Grouped-Query Attention, and head dimension 128 in 16-bit precision), a single batch-1 128K-token sequence ($L = 131{,}072$) requires exactly 16.0 GiB (17.18 GB) of memory purely for the KV cache ($2 \times 32 \times 8 \times 128 \times 131{,}072 \times 2$ bytes)—exceeding the model's static weight footprint and exhausting consumer GPU VRAM.

To address this severe memory wall, the machine learning community has introduced a diverse taxonomy of context compression paradigms:

1. **KV Cache Quantization**: Low-precision numerical formats (FP8, INT8, INT4, INT2) with per-channel or group-wise dynamic scaling (Dettmers et al., 2022; Hooper et al., 2024; Liu et al., 2024).
2. **KV Cache Eviction & Sparsification**: Attention sinks and dynamic heavy-hitter token eviction (e.g., StreamingLLM (Xiao et al., 2023), SnapKV (Li et al., 2024), H_2O (Zhang et al., 2023), Scissorhands (Liu et al., 2023)).
3. **KV Cache Merging & Pooling**: Temporal and semantic clustering of adjacent token representations (Ge et al., 2023; Zhang et al., 2024a).
4. **Low-Rank & Subspace Projection**: SVD and principal-component latent projections (Sharma et al., 2024).
5. **Factorized & Differential State Memory**: Shared low-rank subspace bases with anchor frames and dynamic residuals (Authors withheld for double-blind review, 2026).

Despite this rapid algorithmic progress, an acute methodological challenge exists: *the field lacks a unified, method-agnostic, resource-aware evaluation benchmark.* Current long-context benchmarks (e.g., Needle-In-A-Haystack (Kamradt, 2023), RULER (Hsieh et al., 2024), LongBench (Bai et al., 2023b), LongBench v2 (Bai et al., 2024)) evaluate task accuracy under uncompressed representations without tracking memory consumption. Conversely, compression papers report accuracy on idiosyncratic task subsets alongside theoretical compression ratios, without controlling for base model capability, metadata storage overheads, or hardware latency.

CRBench Overview. **CRBench** resolves these challenges with a query-level, dense-paired evaluation primitive that jointly records task outcomes, memory, and runtime. As illustrated in Figure 1, CRBench decouples quality–memory utility (Part 1, $\mathcal{S}_{\text{res}}$) from hardware system serving utility (Part 2, $\mathcal{S}_{\text{sys}}$).

Method Neutrality. CRBench evaluates all candidate compression methods using a standardized execution and measurement pipeline, with identical query-level pairing, resource accounting, and reporting procedures across methods.

Core Contributions. The primary contributions of this work are:

- **The Query-Level Evaluation Primitive**: We formulate benchmark evaluation around the atomic 4-tuple $(\mathcal{M}, q, \mathcal{R}_{\text{dense}}, \mathcal{A})$, pairing every compressed method evaluation directly against the identical model's uncompressed dense FP16 reference on the exact same prompt.

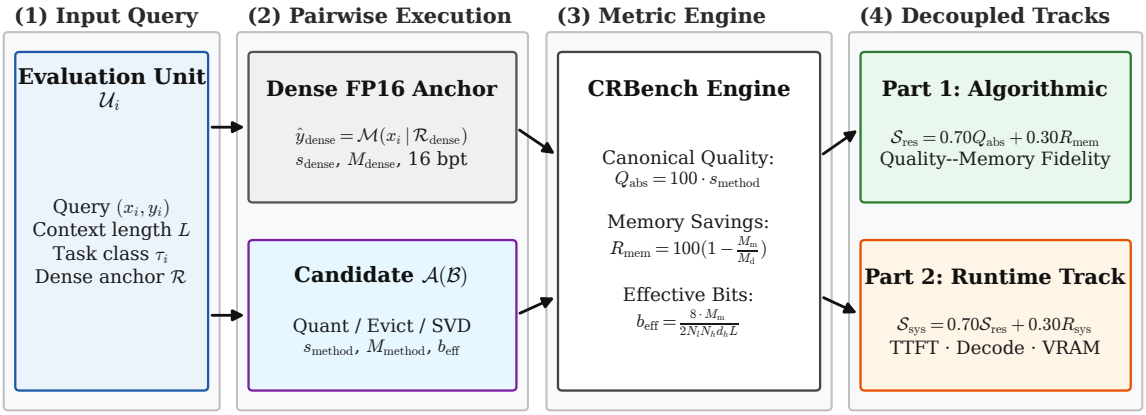


Figure 1: **CRBench Method-Agnostic Evaluation Framework Architecture.** The benchmark decouples evaluation into two complementary tracks: Part 1 (Algorithmic Representation Utility $\mathcal{S}_{\text{res}}$) and Part 2 (Hardware System Track $\mathcal{S}_{\text{sys}}$). Every candidate context representation $\mathcal{A}(\mathcal{B})$ is evaluated pairwise against the base model's uncompressed dense reference $\mathcal{R}_{\text{dense}}$ on the identical input query (x_i, y_i).

- **Auditable Quality and Dense Pairing**: We report canonical absolute task quality $Q_{\text{abs}} \in [0, 100]$ while preserving the paired dense score and paired-execution coverage for every query.
- **Explicit Analytical & Physical Memory Accounting (R_{mem})**: We formalize effective bits-per-element (b_{eff}), accounting not only for raw tensor payloads but also quantization scales, codebooks, eviction index bitmaps, and alignment overheads.
- **Desiderata-Based Scoring Formulation ($\mathcal{S}_{\text{res}}$)**: We audit seven candidate utility formulations against five formal measurement axioms, establishing the linear additive formulation $\mathcal{S}_{\text{res}} = \alpha Q + (1 - \alpha) R_{\text{mem}}$ as a transparent, non-destructive scoring metric.
- **Decoupled Two-Track Architecture**: We decouple algorithmic representation quality (Part 1) from hardware-dependent runtime profiling (Part 2, $\mathcal{S}_{\text{sys}}$), capturing TTFT, decode throughput, peak VRAM, and system multiplier ϕ.
- **Cross-Model Publication Benchmark (3,150 Queries)**: We evaluate seven context-compression methods alongside an uncompressed dense reference across Gemma 4 E2B (2K–128K ladder), Gemma 4 E4B (2K–32K), and Qwen2.5 7B (2K–32K) on NVIDIA RTX hardware.

2 Related Work

2.1 Limitations of Existing Evaluation Paradigms

Existing approaches to evaluating long-context models and KV compression techniques suffer from structural shortcomings, summarized in Table 1:

1. **Quality–Resource Disconnection**: Traditional benchmarks (e.g., L-Eval (An et al., 2023), BAMBOO (Dong et al., 2023), InfiniteBench (Zhang et al., 2024b)) measure accuracy without tracking memory. A method achieving 90% accuracy while consuming 16 bits per token is treated identically to a method achieving 90% accuracy at 4 bits per token, despite the latter offering 4× greater hardware efficiency.
2. **Cross-Model Confounding**: Absolute task accuracy also reflects the base model's native capability. CRBench therefore reports each model separately and logs paired dense outcomes on the identical query, allowing readers to inspect representation effects without presenting a cross-model score as a causal comparison.

Table 1: Comparison of Evaluation Paradigms vs. CRBench. Summary of dimensions captured across long-context benchmarks, synthetic needle tests, inference profilers, and CRBench.

Evaluation Paradigm	Quality	Memory	Latency	Dense Anchor	Pairwise
Long-Context Suites (LongBench, L-Eval)	✓	✗	✗	✗	✗
Synthetic Needle Tests (NIAH, RULER)	✓	✗	✗	✗	✗
Inference Systems Profilers (vLLM)	✗	✓	✓	✗	✗
Ad-hoc Compression Literature	✓	△	△	✗	✗
CRBench (This Work)	✓	✓	✓	✓	✓

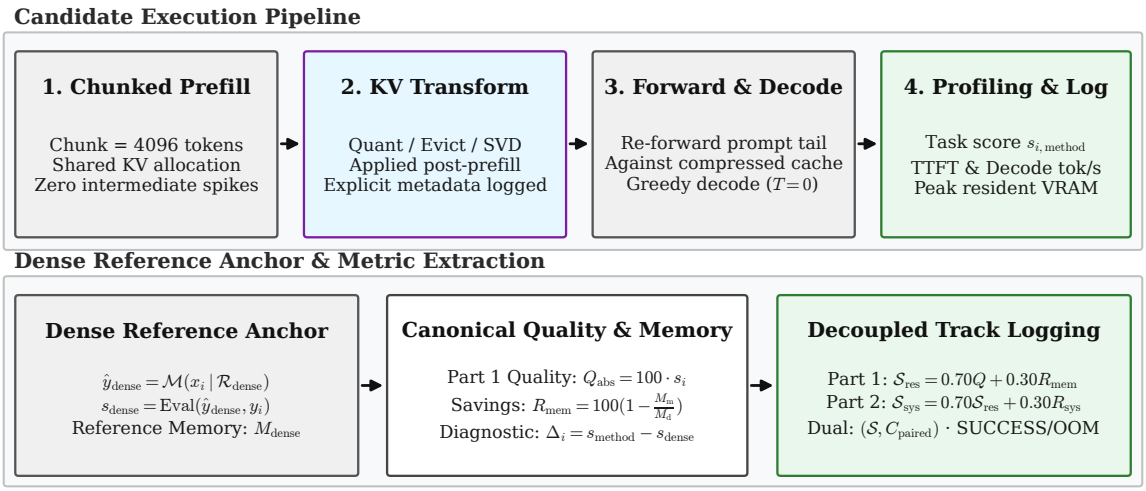


Figure 2: **Atomic Query-Level Evaluation Pipeline in CRBench.** Execution flow from a single query through chunked prefill, transformed-cache re-forwarding, exact metadata tracking, paired dense diagnostics, canonical quality–memory scoring, and dataset-level aggregation.

3. **Omission of Metadata and Runtime Overheads**: Many compression works report theoretical payload bits while omitting the memory required for scaling factors, outlier vectors, or eviction index tables. Furthermore, algorithms that save memory via uncoalesced memory gathers or CPU–GPU synchronization can severely degrade prefill latency (TTFT) and decode throughput.

3 Problem Definition & Mathematical Formulation

3.1 The Atomic Evaluation Primitive

Let $\mathcal{M}$ denote an autoregressive language model parameterized by weights Θ. An evaluation dataset $\mathcal{D} = \{(x_i, y_i, \tau_i, L_i)\}_{i=1}^{N}$ consists of queries x_i, ground-truth target outputs y_i, task types τ_i, and prompt context token lengths L_i.

Let $\mathcal{A}$ denote a candidate context memory adapter (e.g., INT4 quantization or SnapKV eviction) operating under assigned resource budget $\mathcal{B}$. In standard dense inference, the uncompressed reference adapter is denoted $\mathcal{R}_{\text{dense}}$ (16-bit FP16/BF16 representation).

Definition 1 (CRBench Atomic Evaluation Unit)**.** The fundamental evaluation unit in CRBench is the 4-tuple:

$$\mathcal{U}_i = \Big(\mathcal{M},\ (x_i, y_i),\ \mathcal{R}_{\text{dense}},\ \mathcal{A}(\mathcal{B})\Big) \tag{2}$$

For every query (x_i, y_i), CRBench executes:

1. **Dense Reference Execution**: Model $\mathcal{M}$ generates prediction $\hat{y}_{i,\text{dense}} = \mathcal{M}(x_i \mid \mathcal{R}_{\text{dense}})$.
2. **Candidate Adapter Execution**: Model $\mathcal{M}$ generates prediction $\hat{y}_{i,\text{method}} = \mathcal{M}(x_i \mid \mathcal{A}(\mathcal{B}))$.
3. **Quality Measurement**: Raw task scores $s_{i,\text{dense}} = \text{Eval}(\hat{y}_{i,\text{dense}}, y_i) \in [0,1]$ and $s_{i,\text{method}} = \text{Eval}(\hat{y}_{i,\text{method}}, y_i) \in [0,1]$ are recorded.
4. **Resource Measurement**: Memory footprints $M_{i,\text{dense}}$ and $M_{i,\text{method}}$ (in bytes) are computed analytically and tracked physically.

3.2 Canonical Quality and Dense-Reference Diagnostics

The canonical leaderboard deliberately uses an absolute task-success quantity. This makes scores comparable across the full suite and keeps a weak dense baseline from inflating a compressed method's apparent retention. The paired dense execution remains essential: it records the reference outcome, enables failure diagnosis, and defines coverage.

Definition 2 (Canonical Absolute Task Quality)**.** Let $s_{i,\text{method}} \in [0,1]$ be the task evaluator's score for a completed candidate run. Its canonical quality is:

$$Q_{\text{abs},i} = 100.0 \, \min\left(1.0, \max\left(0.0, s_{i,\text{method}}\right)\right). \tag{3}$$

Paired diagnostics and coverage. We log the signed paired difference $\Delta_i = s_{i,\text{method}} - s_{i,\text{dense}}$ and report paired execution coverage $C_{\text{paired}} = N_{\text{paired-success}}/N_{\text{total}}$. Thus, a zero score from a completed run is distinguished from an out-of-memory or runtime failure. Relative retention can be recomputed from the logged pairs for a task with a defensible chance floor, but it is not substituted into the canonical reported scores.

3.3 Memory Resource Accounting (R_{mem})

CRBench enforces explicit state memory accounting. For any adapter $\mathcal{A}$, the total memory consumed by the context representation is partitioned into three additive components:

$$M_{\text{method}}(L) = M_{\text{algo}}(L) + M_{\text{meta}}(L) + M_{\text{align}}(L) \quad \text{[bytes]} \tag{4}$$

where M_{algo} is the raw compressed tensor payload, M_{meta} represents all mandatory auxiliary metadata (quantization scale factors, zero-point offsets, codebook dictionaries, or 32-bit integer token position index arrays for eviction methods), and M_{align} represents hardware memory alignment and padding overheads.

Definition 3 (Effective Bits Per KV Element)**.** The effective bits-per-element (b_{eff}) measures the total memory footprint normalized by the total uncompressed Key-Value tensor scalar count ($2 \times N_{\text{layers}} \times N_{\text{kv_heads}} \times d_{\text{head}} \times L$):

$$b_{\text{eff}} = \frac{8 \cdot M_{\text{method}}(L)}{2 \cdot N_{\text{layers}} \cdot N_{\text{kv_heads}} \cdot d_{\text{head}} \cdot L} \quad \text{[bits/element]} \tag{5}$$

For standard uncompressed 16-bit float KV caches (FP16/BF16), $b_{\text{eff}} = 16.0$ bits/element.

Definition 4 (Memory Resource Savings)**.** The normalized memory resource savings $R_{\text{mem},i} \in [0.0, 100.0]$ is:

$$R_{\text{mem},i} = 100.0 \times \max\left(0.0,\ 1.0 - \frac{M_{i,\text{method}}}{M_{i,\text{dense}}}\right) \tag{6}$$

4 CRBench Scoring Formulation

4.1 Measurement Desiderata for Utility Formulation

A useful scalar utility must be monotone in both absolute task quality and resource savings, bounded on $[0,100]^2$, and Pareto consistent. The linear utility below satisfies these properties directly; its short proof and a comparison with alternative formulations are deferred to the supplementary material.

4.2 Part 1: Resource Utility Score ($\mathcal{S}_{\text{res}}$)

CRBench defines the canonical Part 1 Resource Score using a linear additive utility formulation:

$$\mathcal{S}_{\text{res},i} = \alpha \cdot Q_{\text{abs},i} + (1 - \alpha) \cdot R_{\text{mem},i} \tag{7}$$

where $\alpha \in [0.10, 0.90]$ is the global quality preference weight, with standard configuration $\alpha = 0.70$.

Default trade-off. The $\alpha = 0.70$ configuration makes a one-point gain in task quality worth 2.33 points of memory savings. A dense run with perfect task success has score 70.0, whereas a 4-bit method at 94% task quality and 75% savings has score 88.3. The sensitivity analysis is supplied in the supplement.

4.3 Dataset Aggregation & Coverage Reliability (C_{paired})

For an evaluation dataset $\mathcal{D}$, the aggregate benchmark score is computed by averaging query-level utility scores:

$$\bar{\mathcal{S}}_{\text{res}} = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} \mathcal{S}_{\text{res},i}, \quad \bar{Q}_{\text{abs}} = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} Q_{\text{abs},i}, \quad \bar{R}_{\text{mem}} = \frac{1}{|\mathcal{D}|} \sum_{i=1}^{|\mathcal{D}|} R_{\text{mem},i} \tag{8}$$

CRBench computes 95% non-parametric bootstrap confidence intervals with $B = 2,000$ resamples.

Coverage Decoupling. CRBench reports paired query execution coverage $C_{\text{paired}} = \frac{N_{\text{paired_success}}}{N_{\text{total}}} \in [0.0, 1.0]$ alongside $\mathcal{S}_{\text{res}}$ as a dual-metric pair $(\mathcal{S}_{\text{res}}, C_{\text{paired}})$, rather than multiplicatively folding coverage into $\mathcal{S}_{\text{res}}$. This preserves algorithmic evaluation integrity even if occasional memory allocator faults occur.

5 System Evaluation Track (Part 2)

While Part 1 evaluates algorithmic memory representation efficiency, practical deployment requires analyzing physical latency and throughput. An adapter that compresses tensors effectively but relies on un-fused loops or irregular memory gathers can suffer severe latency degradation.

5.1 Profiled System Metrics

CRBench incorporates hardware-synchronized wall-clock timers and device memory allocators to capture:

1. **Time-To-First-Token (TTFT)**: Total wall-clock time (in seconds) required to process prompt context of length L and emit the first token.
2. **Autoregressive Decode Throughput ($\mathcal{T}_{\text{dec}}$)**: Generation speed in tokens per second (tok/s):

$$\mathcal{T}_{\text{dec}} = \frac{N_{\text{generated}}}{\Delta t_{\text{decode}}} \quad [\text{tok/s}] \tag{9}$$

3. **Peak Resident Device VRAM ($\mathcal{M}_{\text{peak}}$)**: Maximum allocated and reserved GPU memory during inference.

5.2 System Efficiency and Part 2 Score ($\mathcal{S}_{\text{sys}}$)

To evaluate real-world deployment viability under physical hardware constraints, CRBench computes normalized efficiency components relative to the uncompressed dense baseline:

$$\phi_{\text{ttft}} = \min\left(1.0, \frac{\text{TTFT}_{\text{dense}}}{\text{TTFT}_{\text{method}}}\right), \quad \phi_{\text{thru}} = \min\left(1.0, \frac{\mathcal{T}_{\text{method}}}{\mathcal{T}_{\text{dense}}}\right), \quad \phi_{\text{vram}} = \min\left(1.0, \frac{\mathcal{M}_{\text{dense}}}{\mathcal{M}_{\text{method}}}\right) \tag{10}$$

These three physical dimensions are integrated into a composite hardware efficiency multiplier $\phi \in [0.0, 1.0]$ via a geometric mean:

$$\phi = \phi_{\text{ttft}}^{0.35} \cdot \phi_{\text{thru}}^{0.35} \cdot \phi_{\text{vram}}^{0.30}, \quad \text{yielding} \quad R_{\text{sys}} = 100.0 \times \phi \tag{11}$$

where weights allocate balanced emphasis to interactive responsiveness (TTFT 35%, decode throughput 35%) and physical resident device footprint (VRAM 30%). All system-efficiency components and $\mathcal{S}_{\text{sys}}$ are computed at the individual-query level prior to aggregation; the TTFT, throughput, and VRAM values reported in the tables are descriptive arithmetic means and are not substituted directly into the composite formula.

The composite **CRBench Part 2 System Serving Score** ($\mathcal{S}_{\text{sys}} \in [0.0, 100.0]$) then combines the algorithmic representation utility ($\mathcal{S}_{\text{res}}$) from Part 1 with physical system efficiency (R_{sys}) using the canonical utility formulation:

$$\mathcal{S}_{\text{sys}} = \alpha \cdot \mathcal{S}_{\text{res}} + (1 - \alpha) \cdot R_{\text{sys}} \quad (\text{with } \alpha = 0.70) \tag{12}$$

Methods that meet or exceed the dense baseline on TTFT and decode throughput without exceeding its VRAM footprint achieve $\phi = 1.0$ ($R_{\text{sys}} = 100.0$), while methods incurring prefill stalls or memory bloat receive hardware penalties ($\phi < 1.0$) that rigorously discount their serving score.

6 Experimental Setup

6.1 Evaluated Models & Context Ladders

We evaluate three open-weight foundation models across 3,150 standardized queries:

- **Gemma 4 E2B**: Evaluated across the complete 2K $\rightarrow$ 128K context ladder (2K, 8K, 32K, 64K, 128K tokens, 1,050 queries).
- **Gemma 4 E4B**: Evaluated across context horizons from 2K to 32K (2K, 4K, 8K, 16K, 32K tokens, 1,050 queries).
- **Qwen2.5 7B**: Evaluated across context horizons from 2K to 32K (2K, 4K, 8K, 16K, 32K tokens, 1,050 queries).

6.2 Evaluated Methods

We evaluate seven representative context compression methods alongside the uncompressed 16-bit dense reference:

1. **KIVI-style INT4 Quantization**: 4-bit asymmetric per-channel keys and 4-bit per-token values following the grouping scheme of KIVI (Liu et al., 2024).
2. **SnapKV**: Observation-guided heavy-hitter eviction ($L_{\text{obs}} = 32, k = 7$) (Li et al., 2024).
3. **DKV-Mid / High** (Authors withheld for double-blind review, 2026): Differential KV with 128- / 256-rank subspace and dynamic residuals.
4. **Low-Rank KV**: Per-head mean-centered truncated SVD projection (Sharma et al., 2024).
5. **StreamingLLM**: Attention sink ($N_{\text{sink}} = 4$) + rolling local window eviction (Xiao et al., 2023).
6. **KV-Merging**: Temporal adjacent token mean-pooling (Ge et al., 2023).
7. **Dense FP16**: Full uncompressed 16-bit reference baseline ($b_{\text{eff}} = 16.0$ bpt).

6.3 Evaluation Tasks

1. **Single & Multi NIAH**: Single- and distributed multi-target needle retrieval at parameterized insertion depths.
2. **RULER KV**: High-entropy dictionary key-value lookup amid distracting pairs (Hsieh et al., 2024).
3. **Variable Tracking**: Multi-step variable reassignment tracking (Hsieh et al., 2024).
4. **Multi-Hop QA**: Cross-document synthetic multi-hop reasoning.

Table 2: **Cross-Model Part 1 ($\mathcal{S}_{\text{res}}$) Leaderboard.** Evaluated under canonical Option B physical memory aggregation across 3,150 queries. Best score per model in **bold**.

	Gemma 4 E2B (2K–128K)			Gemma 4 E4B (2K–32K)			Qwen2.5 7B (2K–32K)		
Method	**Accuracy**	$b_{\text{eff}}^{\text{agg}}$	$\mathcal{S}_{\text{res}}$	**Accuracy**	$b_{\text{eff}}^{\text{agg}}$	$\mathcal{S}_{\text{res}}$	**Accuracy**	$b_{\text{eff}}^{\text{agg}}$	$\mathcal{S}_{\text{res}}$
KIVI (INT4)	46.4%	6.01 bpt	51.2	62.0%	6.04 bpt	**62.1**	60.7%	6.04 bpt	**61.1**
SnapKV	46.8%	6.00 bpt	**51.5**	60.0%	6.00 bpt	60.8	58.2%	6.00 bpt	59.5
DKV-Mid	44.7%	5.46 bpt	51.0	55.3%	5.00 bpt	59.4	53.3%	5.84 bpt	56.3
DKV-High	46.7%	10.78 bpt	42.5	61.3%	10.68 bpt	52.9	59.1%	12.14 bpt	48.6
Dense FP16	46.4%	16.00 bpt	32.5	64.0%	16.00 bpt	44.8	60.0%	16.00 bpt	42.0
Low-Rank KV	31.2%	6.07 bpt	40.5	48.8%	6.27 bpt	52.4	28.6%	6.09 bpt	38.6
StreamingLLM	22.9%	6.00 bpt	34.8	25.5%	6.00 bpt	36.6	36.5%	6.00 bpt	44.3
KV-Merging	13.3%	6.01 bpt	28.0	25.3%	6.03 bpt	36.4	19.9%	6.03 bpt	32.6

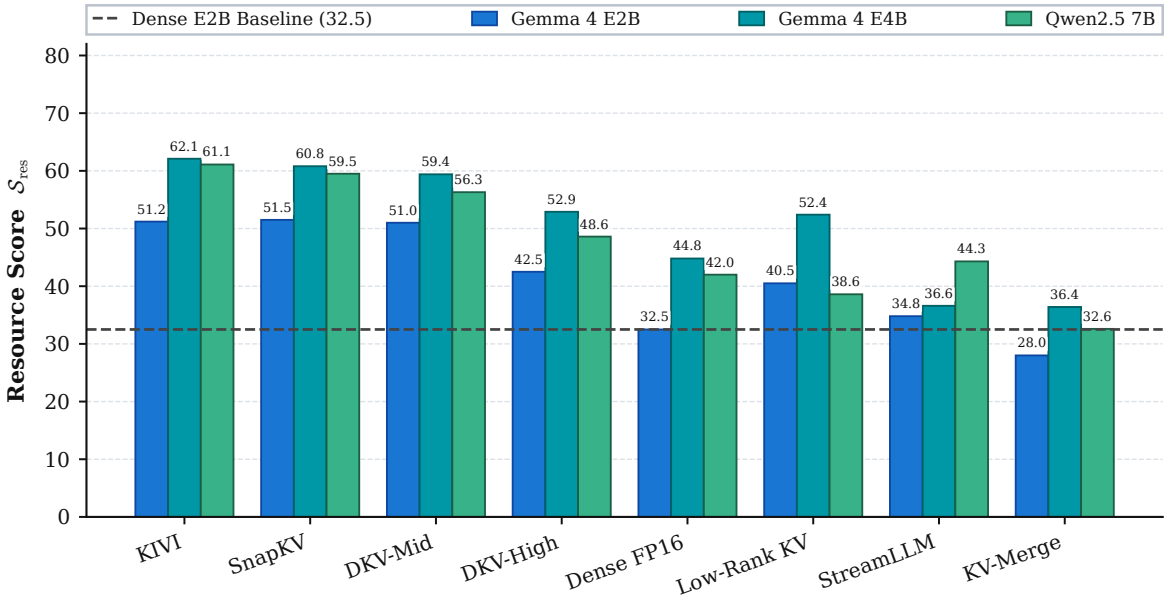


Figure 3: **Cross-Model Part 1 Resource Utility ($\mathcal{S}_{\text{res}}$) Comparison.** Evaluated across Gemma 4 E2B, Gemma 4 E4B, and Qwen2.5 7B.

7 Results

7.1 Cross-Model Algorithmic Representation Fidelity ($\mathcal{S}_{\text{res}}$)

Table 2 presents the Part 1 leaderboard across all three models under canonical scoring $\mathcal{S}_{\text{res}} = 0.70 \cdot Q_{\text{abs}} + 0.30 \cdot R_{\text{mem}}^{\text{agg}}$.

7.2 Cross-Model Hardware System Serving Utility ($\mathcal{S}_{\text{sys}}$)

Table 3 presents the Part 2 system scores. The per-model distribution is provided in the supplementary material.

Table 3: **Cross-Model Part 2 Hardware System Serving Utility ($\mathcal{S}_{\text{sys}}$).** Ranked by cross-model mean score.

Method	Gemma 4 E2B	Gemma 4 E4B	Qwen2.5 7B	Cross-Model Mean $\mathcal{S}_{\text{sys}}$
KIVI (INT4)	64.6	73.4	72.8	**70.3**
DKV-Mid	59.1	66.3	54.2	**59.9**
Low-Rank KV	57.4	65.9	54.9	**59.4**
Dense FP16	52.8	61.4	59.4	**57.8**
StreamingLLM	54.3	55.6	61.0	**57.0**
SnapKV	60.6	48.6	53.7	**54.3**
DKV-High	52.7	60.0	48.5	**53.7**
KV-Merging	49.6	55.5	52.8	**52.6**

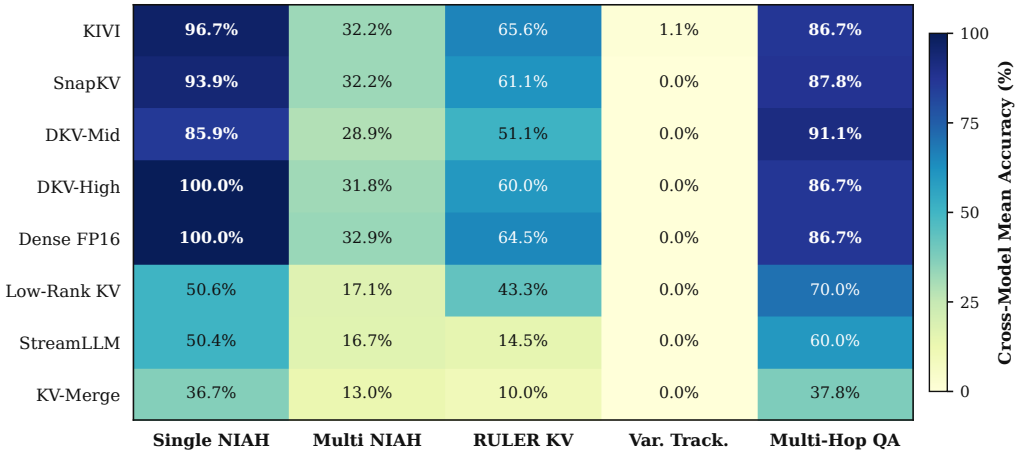


Figure 4: **Task-Wise Accuracy Heatmap Across Compression Paradigms.** Cross-model average accuracy (%) across all three evaluated models.

7.3 Task-Wise Performance Breakdown

Figure 4 visualizes the cross-model mean accuracy by task archetype.

7.4 Ultra-Long Context Ladder (2K $\rightarrow$ 128K) Scaling

Figure 5 traces performance across the full 2K–128K context ladder on Gemma 4 E2B, illustrating the degradation cliff beyond 64K tokens.

7.5 Empirical Quality–Memory Pareto Frontier

Figure 6 maps task accuracy against effective bits-per-element b_{eff}, highlighting the non-dominated Pareto frontier across all three models.

8 Analysis & Discussion

KIVI Dominates System Serving ($\bar{\mathcal{S}}_{\text{sys}} = 70.3$). KIVI-style INT4 quantization achieves the highest cross-model system score because asymmetric quantization operates during cache writes without prefill latency spikes, maintaining $\phi \geq 0.96\times$ across all models.

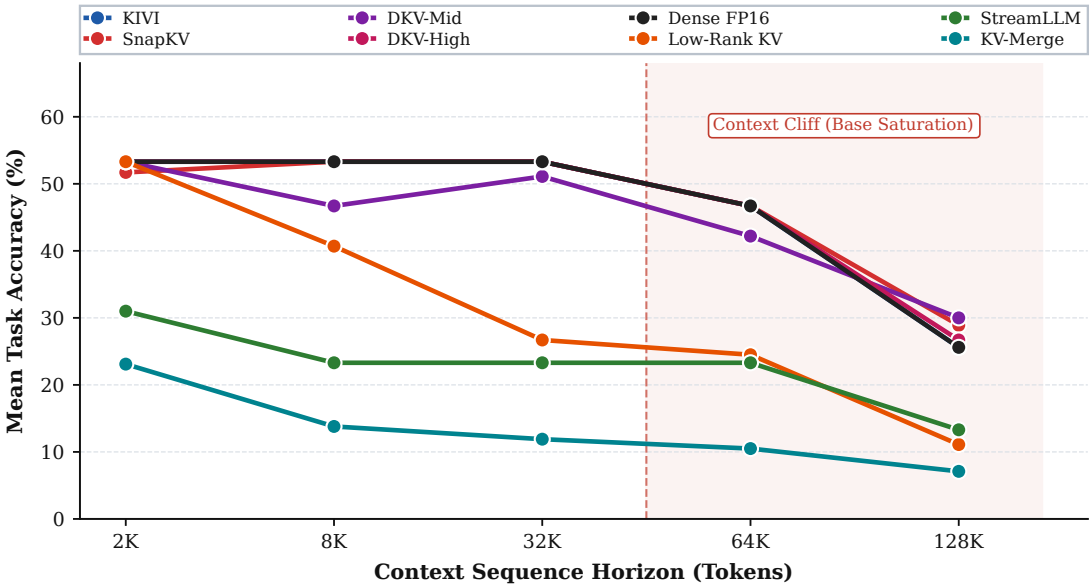


Figure 5: **Ultra-Long Context Ladder Scaling (2K to 128K Tokens) on Gemma 4 E2B.** Accuracy averaged across all 5 tasks, showing sharp degradation at 64K–128K shared with the uncompressed baseline.

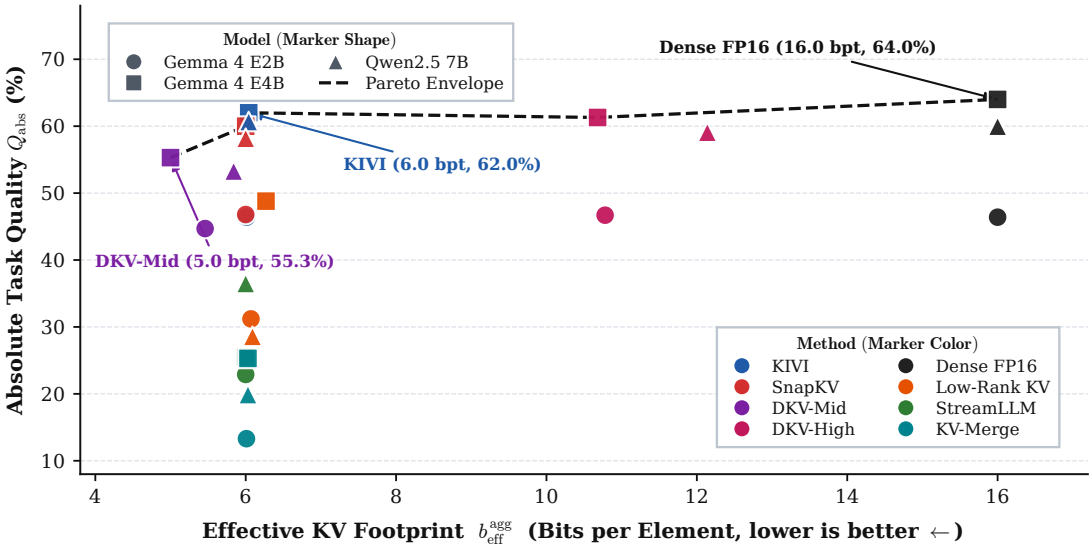


Figure 6: **Quality–Memory Pareto Frontier Across 3 Foundation Models.** Evaluated across 24 configurations, mapping task accuracy against effective bits-per-element b_{eff}. Marker shapes denote foundation model (circle: Gemma 4 E2B, square: Gemma 4 E4B, triangle: Qwen2.5 7B); marker colors denote compression method.

SnapKV's Prefill Hardware Penalty. While SnapKV attains high algorithmic utility ($\mathcal{S}_{\text{res}} = 60.8$ on E4B), un-fused observation pooling on 32K sequences inflates TTFT to 83.4s and peak VRAM to 2,632 MB ($\phi = 0.20\times$), dropping its Part 2 score to 48.6 and confirming the necessity of decoupled system profiling.

DKV-Mid Pushes the Memory Frontier (5.00–5.84 bpt). DKV-Mid operates at low bitrates (5.00 bpt on E4B) and attains the highest cross-model Multi-Hop QA accuracy in our evaluated suite (91.1%). However, SVD projection increases TTFT (22.3s on E2B, 39.6s on 7B), resulting in $\phi \approx 0.49$–$0.83\times$.

Variable Tracking: Model-Level Limitation. All evaluated methods, including uncompressed dense FP16, score near 0.0% on Variable Tracking, indicating that the evaluated models have a substantial capability limitation on this task that is not explained by KV compression alone.

9 Limitations & Research Scope

We identify four key operational boundaries: (1) **Model Scale**: Current evaluation benchmarks 2B to 7B architectures; future extensions will evaluate 13B+ and 70B+ model families. (2) **Task Coverage**: Our suite prioritizes long-context needle retrieval, associative lookup, and multi-hop reasoning; open-ended agentic dialogues and summarization represent natural next steps. (3) **Hardware Scope**: Profiling is conducted on single-node workstation GPUs; distributed multi-GPU serving may alter Part 2 communication costs. (4) **Kernel Fusion**: Algorithms evaluated via pure PyTorch incur prefill memory and dispatch latency penalties that dedicated fused CUDA/Triton kernels can substantially mitigate.

10 Conclusion

We presented **CRBench**, a method-agnostic, resource-aware evaluation framework for long-context LLMs. By anchoring evaluations pairwise against the uncompressed dense reference, enforcing explicit memory accounting via b_{eff}, and decoupling algorithmic utility ($\mathcal{S}_{\text{res}}$) from hardware serving ($\mathcal{S}_{\text{sys}}$), CRBench establishes a rigorous foundation for evaluating context memory representations.

Broader Impact Statement

Context representation and compression technologies directly influence the economic accessibility and environmental footprint of frontier artificial intelligence. By reducing KV cache memory demands by up to $3.2\times$, efficient context representations democratize long-context serving on commodity hardware and substantially decrease data center energy consumption and carbon emissions. However, aggressive eviction and lossy quantization can induce behavioral drift, hallucinations, or silent omission of needle facts in high-stakes domains (e.g., clinical retrieval or legal contract analysis). CRBench directly mitigates these risks by providing an auditable, multi-dimensional evaluation suite that rigorously exposes failure modes and hardware bottlenecks before compression algorithms are deployed in production systems.

References

Achiam, J., Adler, S., Agarwal, S., et al. (2023). GPT-4 technical report. *arXiv preprint arXiv:2303.08774*.

An, C., Gong, S., Zhong, M., et al. (2023). L-Eval: Instituting standardized evaluation for long context language models. *arXiv preprint arXiv:2307.11088*.

Bai, J., Bai, S., Chu, Y., et al. (2023a). Qwen technical report. *arXiv preprint arXiv:2309.16609*.

Bai, Y., Lv, X., Zhang, J., et al. (2023b). LongBench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*.

Bai, Y., Lv, X., Zhang, J., et al. (2024). LongBench v2: Towards deeper understanding of long context large language models. *arXiv preprint arXiv:2412.15204*.

Brown, T., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. In *NeurIPS*, vol. 33, pp. 1877–1901.

[Authors withheld for double-blind review] (2026). DKV: Anchor + Low-Rank Differential KV-Cache Compression for Scalable Long-Context Inference. *Preprint under double-blind review.*

DeepSeek-AI (2024). DeepSeek-V2: A strong, economical, and efficient Mixture-of-Experts language model. *arXiv preprint arXiv:2405.04434*.

Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. (2022). LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *NeurIPS*, vol. 35, pp. 30318–30332.

Dong, Z., Tang, T., Dai, J., et al. (2023). BAMBOO: A comprehensive benchmark for evaluating long-sequence language models. *arXiv preprint arXiv:2309.13345*.

Ge, S., Zhang, Y., Liu, L., et al. (2023). Model tells you what to discard: Adaptive KV cache compression for LLMs. *arXiv preprint arXiv:2310.01801*.

Hooper, C., Kim, S., Mohammadzadeh, H., et al. (2024). KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *NeurIPS*.

Hsieh, C.-Y., Sun, S., Kriman, S., et al. (2024). RULER: What's the real context size of your long-context language model? *arXiv preprint arXiv:2404.06654*.

Kamradt, G. (2023). Needle in a haystack - pressure testing LLMs. *GitHub repository: gkamradt/LLMTest_NeedleInAHaystack*.

Li, Y., Huang, Y., Yang, B., et al. (2024). SnapKV: LLM knows what you are looking for before generation. In *NeurIPS*.

Liu, Z., Desai, A., Liao, F., et al. (2023). Scissorhands: Exploiting the persistence of importance in LLM KV cache. In *NeurIPS*.

Liu, Z., Yuan, J., Jin, H., et al. (2024). KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *ICML*.

Reid, M., Savinov, N., Teplyashin, D., et al. (2024). Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*.

Sharma, P., Ash, J. T., and Misra, D. (2024). The truth is in there: Improving reasoning in language models with layer-selective rank reduction. In *ICLR*.

Touvron, H., Lavril, T., Izacard, G., et al. (2023). LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. In *NeurIPS*, vol. 30, pp. 5998–6008.

Xiao, G., Tian, Y., Chen, B., et al. (2023). Efficient streaming language models with attention sinks. In *ICLR*.

Zhang, Z., Sheng, Y., Zheng, T., et al. (2023). H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*.

Zhang, Y., Du, X., Liu, C., et al. (2024a). PyramidKV: Dynamic KV cache compression based on pyramid information funneling. *arXiv preprint arXiv:2406.02069*.

Zhang, X., Chen, Y., Hu, S., et al. (2024b). ∞-Bench: Extending context window to 100K+ for comprehensive evaluation of LLMs. In *ACL*.